

Lembar Materi & Praktikum: Pemrograman Konkuren dengan Goroutines, Channels, dan Sinkronisasi di Golang Gin

Durasi: 3 Jam (180 Menit) **Level:** Advanced (Lanjut)

Praktikum ini melengkapi materi teori pada [21. concurrent-api.md](#). Anda akan dipandu untuk mengimplementasikan pemrograman konkuren (*concurrency*) di bahasa Go pada proyek **Sistem Tiket Konser** (`go-tiket-konser`) dengan framework **Gin**. Kita akan belajar memicu tugas asinkron latar belakang menggunakan **Goroutines**, mengamankan variabel memori bersama dari *Race Condition* menggunakan **sync.Mutex/sync.RWMutex**, menyinkronkan siklus hidup goroutine dengan **sync.WaitGroup**, berkomunikasi secara aman melalui **Channels**, serta membangun pola arsitektur **Worker Pool** teroptimasi di dalam Handler API Gin.

Sesi 1: Teori Dasar Konkurensi & Goroutines (45 Menit)

1. Konkurensi vs Paralelisme

- **Konkurensi (Concurrency):** Struktur pemrograman yang menangani banyak tugas sekaligus secara bergantian (*dealing with lots of things at once*). Contoh: Satu kasir melayani antrean pembeli A dan pembeli B secara bergantian dengan cepat.
- **Paralelisme (Parallelism):** Eksekusi fisik dari banyak tugas secara bersamaan pada waktu yang persis sama memanfaatkan banyak CPU Core (*doing lots of things at once*). Contoh: Dua kasir melayani pembeli A dan B masing-masing secara bersamaan.

NOTE

Go dirancang untuk menangani konkurensi dengan sangat baik di tingkat bahasa melalui penjadwal bawaan (*Go Runtime Scheduler*).

2. Mengapa Goroutine Sangat Efisien?

Di bahasa pemrograman tradisional (seperti Java atau C++), pemrograman konkuren dilakukan dengan membuat *Operating System (OS) Thread*.

- **OS Thread:** Membutuhkan alokasi memori awal yang besar (~2 Megabyte per thread), pemindahan konteks (*context switch*) lambat karena melibatkan kernel OS.

- **Goroutine:** Dikelola sepenuhnya oleh Go Runtime (bukan OS kernel), membutuhkan alokasi memori awal yang sangat kecil (**hanya ~2 Kilobyte**), dan context switch terjadi sangat cepat di ruang pengguna (*user space*). Anda dapat menjalankan ratusan ribu goroutine sekaligus tanpa membebani memori server.

3. Keyword `go` dan Celah Keamanan Data (Race Condition)

Menjalankan fungsi secara asinkron di Go sangat mudah, cukup menambahkan kata kunci `go` di depan pemanggilan fungsi:

```
go KirimEmailAsinkron()
```

Namun, kemudahan ini memunculkan risiko fatal bernama **Race Condition**: yaitu bug ketika dua atau lebih Goroutine mencoba membaca dan memodifikasi satu lokasi memori variabel yang sama secara bersamaan tanpa pelindung.

- *Contoh:* Sisa tiket konser bernilai 1. Klien A dan B menekan tombol beli secara bersamaan. Kedua goroutine membaca sisa tiket bernilai 1, keduanya lolos validasi, stok berkurang menjadi -1 (terjadi penjualan berlebih atau *overselling*).

Sesi 2: Sinkronisasi Terpusat: `sync.WaitGroup` & `sync.Mutex` (45 Menit)

Go menyediakan pustaka standar `sync` untuk menjinakkan perilaku goroutine asinkron.

1. `sync.WaitGroup` (Sinkronisasi Aliran Kerja)

Saat fungsi utama `main()` selesai dieksekusi, seluruh goroutine latar belakang akan dihentikan paksa meskipun tugas mereka belum selesai. Kita menggunakan `sync.WaitGroup` untuk memaksa program menunggu seluruh goroutine selesai melapor.

- `Add(n)` : Menambahkan jumlah goroutine yang harus ditunggu.
- `Done()` : Dipanggil di dalam goroutine setelah tugas selesai (mengurangi antrian).
- `Wait()` : Memblokir eksekusi baris kode berikutnya sampai seluruh tugas melapor `Done()` .

2. `sync.Mutex` (Gembok Eksklusif)

Untuk mencegah *Race Condition*, kita menggunakan `sync.Mutex` (*Mutual Exclusion*). Sebelum variabel bersama dimodifikasi (*Critical Section*), kita memasang gembok `.Lock()` . Goroutine lain yang mencoba mengakses memori tersebut dipaksa mengantre sampai gembok dibuka menggunakan `.Unlock()` .

```
var mu sync.Mutex
var stock = 10

func BuyTicket() {
    mu.Lock()           // Gembok dipasang
    defer mu.Unlock()   // Buka gembok saat fungsi selesai

    if stock > 0 {
        stock--
    }
}
```

3. sync.RWMutex (Gembok Baca-Tulis)

Jika aplikasi kita memiliki intensitas pembacaan data yang tinggi namun jarang terjadi penulisan (seperti katalog detail konser), `sync.Mutex` biasa akan menurunkan performa karena memblokir sesama pembaca.

- **sync.RWMutex** memisahkan jenis penguncian:
 - `RLock()` / `RUnlock()` : Diizinkan diakses oleh banyak Goroutine secara bersamaan khusus untuk **membaca** data.
 - `Lock()` / `Unlock()` : Gembok eksklusif penuh untuk **menulis/mengubah** data. Tidak boleh ada pembaca maupun penulis lain yang aktif saat gembok ini terpasang.

Sesi 3: Komunikasi Antar-Goroutine: Channels & Select (45 Menit)

"Jangan berkomunikasi dengan berbagi memori (Mutex); melainkan berbagilah memori dengan cara berkomunikasi (Channels)."

1. Apa itu Channel?

Goroutine berjalan secara independen dan tidak mengembalikan nilai (*return value*) ke fungsi pemanggilnya. **Channel** bertindak sebagai pipa komunikasi tipe-aman untuk mengirim dan menerima data antar goroutine.

- **Unbuffered Channel** (Default): Mengirim data akan memblokir goroutine pengirim sampai ada goroutine lain yang siap menerima data tersebut dari channel.
- **Buffered Channel**: Memiliki kapasitas penyimpanan sementara (antrean). Pengirim tidak akan terblokir sampai kapasitas buffer tersebut terisi penuh.

```
ch := make(chan string, 5) // Buffered channel dengan kapasitas 5
```

2. Iterasi dan Penutupan Channel

- Mengirim data: `ch ← "data"`
- Menerima data: `data := ←ch`
- **Penutupan Channel:** Menggunakan `close(ch)` untuk memberi tahu penerima bahwa tidak akan ada data baru lagi yang dikirim.
- **Loop Channel:** Membaca isi channel terus-menerus sampai channel tersebut ditutup:

```
for msg := range ch {  
    fmt.Println(msg)  
}
```

3. Multiplexing dengan Select & Timeout

Kata kunci `select` memungkinkan goroutine menunggu beberapa operasi channel sekaligus. `select` akan mengeksekusi *case* channel mana saja yang mengirimkan data paling cepat. Kita juga bisa memanfaatkan `select` dikombinasikan dengan `time.After` untuk mencegah goroutine menggantung selamanya (*Timeout*).

```
select {  
case msg := ←ch:  
    fmt.Println("Menerima:", msg)  
case ←time.After(2 * time.Second):  
    fmt.Println("Koneksi timeout!")  
}
```

Sesi 4: Laboratorium Praktikum Mandiri – Implementasi Worker Pool untuk Ekspor Data di Gin (45 Menit)

Kita akan membuat fitur backend Gin untuk memproses ekspor laporan data pemesanan tiket konser massal. Jika laporan diekspor satu per satu (sekuensial), respons HTTP akan sangat lambat. Kita akan menyewa 3 "Pekerja" (*Worker Goroutines*) untuk memproses antrian laporan secara paralel.

1. Membuat Handler Pemrosesan Worker Pool: [handler/concert_export.go](#)

Buat berkas baru bernama `concert_export.go` di bawah folder `handler/` untuk memproses antrian ekspor secara paralel menggunakan Worker Pool:

```
package handler  
  
import (
```

```

        "fmt"
        "net/http"
        "time"

        "github.com/gin-gonic/gin"
    )

    // ExportJob merepresentasikan tugas ekspor yang harus dikerjakan
    type ExportJob struct {
        BookingID int
        Format     string
    }

    // ExportResult menyimpan hasil laporan setelah diproses
    type ExportResult struct {
        BookingID int
        Status     string
        Duration  time.Duration
        WorkerID  int
    }

    // exportWorker adalah Goroutine pekerja yang terus membaca antrean tugas dari channel
    jobs
    func exportWorker(id int, jobs ←chan ExportJob, results chan← ExportResult) {
        for job := range jobs {
            startTime := time.Now()

            // Simulasi proses kompilasi ekspor laporan PDF/Excel yang berat ke
            Cloud Storage
            fmt.Printf("[Worker %d] Mulai memproses PDF untuk Booking ID:
            %d...\n", id, job.BookingID)
            time.Sleep(500 * time.Millisecond)

            duration := time.Since(startTime)

            // Kirim hasil pengerjaan ke channel results
            results ← ExportResult{
                BookingID: job.BookingID,
                Status:      "Success",
                Duration:  duration,
                WorkerID:  id,
            }
        }
    }

    type ConcertExportHandler struct{}

    func NewConcertExportHandler() *ConcertExportHandler {
        return &ConcertExportHandler{}
    }

    // ExportBookingsAPI memproses massal 15 ekspor laporan pemesanan tiket menggunakan
    pola Worker Pool

```

```

func (h *ConcertExportHandler) ExportBookingsAPI(c *gin.Context) {
    startTime := time.Now()

    totalJobs := 15
    numWorkers := 3 // Mempekerjakan 3 goroutine pekerja secara paralel

    // Inisialisasi buffered channels
    jobs := make(chan ExportJob, totalJobs)
    results := make(chan ExportResult, totalJobs)

    // 1. Jalankan 3 pekerja asinkron (Worker Goroutines)
    for w := 1; w ≤ numWorkers; w++ {
        go exportWorker(w, jobs, results)
    }

    // 2. Masukkan 15 tugas ekspor ke dalam channel jobs
    for j := 1; j ≤ totalJobs; j++ {
        jobs ← ExportJob{
            BookingID: 1000 + j,
            Format:     "PDF",
        }
    }
    close(jobs) // Tutup channel jobs untuk memberi tanda pada pekerja bahwa tidak
    ada tugas baru lagi

    // 3. Kumpulkan hasil pengerjaan dari channel results
    var reports []string
    for r := 1; r ≤ totalJobs; r++ {
        res := ←results
        reports = append(reports, fmt.Sprintf("Booking #%d diproses sukses
oleh Pekerja-%d dalam waktu %v", res.BookingID, res.WorkerID, res.Duration))
    }

    totalDuration := time.Since(startTime)

    // Kembalikan respons JSON dengan data statistik durasi pemrosesan
    c.JSON(http.StatusOK, gin.H{
        "success":      true,
        "message":       "Ekspor massal berhasil diproses menggunakan Worker
Pool secara paralel",
        "total_jobs":    totalJobs,
        "active_workers": numWorkers,
        "elapsed_time":   totalDuration.String(),
        "reports":        reports,
    })
}

```

Daftarkan rute ekspor laporan ini pada file setup routing utama `routes/routes.go` :

```
concertExportHandler := handler.NewConcertExportHandler()
```

```
// POST /api/v1/bookings/export - Pemicu ekspor laporan massal
api.POST("/bookings/export", concertExportHandler.ExportBookingsAPI)
```

Jika dijalankan secara sekuensial biasa tanpa goroutine, ekspor 15 data membutuhkan durasi 7,5 detik ($15 \times 0.5s$). Namun, dengan mempekerjakan 3 worker secara paralel, proses akan selesai hanya dalam waktu sekitar 2,5 detik saja ($3\times$ lipat lebih cepat).

Sesi 5: Tantangan Praktikum Mandiri (Tugas Mandiri)

Selesaikan tantangan di bawah ini untuk melatih keterampilan Anda dalam mendeteksi bug konkurensi dan merancang pembatasan koneksi.

Tantangan 1: Deteksi Race Condition dengan Command `go test -race`

Menemukan bug perebutan data memori (*race condition*) secara manual sangatlah sulit. Go menyediakan tool analyzer bawaan compiler yang sangat andal untuk mendeteksi race condition secara otomatis.

- **Tugas:**

1. Buat file testing baru bernama `service/concurrency_test.go` di root atau sub-folder proyek.
2. Tulis unit test sederhana yang memicu modifikasi variabel counter secara konkuren tanpa menggunakan Mutex:

```
package service

import (
    "sync"
    "testing"
)

func TestUnsafeCounter(t *testing.T) {
    counter := 0
    var wg sync.WaitGroup

    // Picu 100 goroutine untuk menambah nilai counter secara bersamaan
    // tanpa mutex
    for i := 0; i < 100; i++ {
        wg.Add(1)
        go func() {
            defer wg.Done()
            counter++ // Terjadi race condition di sini!
        }()
    }
}
```

```
    wg.Wait()  
}
```

3. Buka terminal proyek dan jalankan pengujian unit menggunakan flag `-race` :

```
go test -race ./service/...
```

4. Amati terminal Anda. Perhatikan bagaimana Go compiler memberikan laporan visual terperinci (*WARNING: DATA RACE*) yang menunjukkan baris kode persis di mana konflik pembacaan/penulisan memori terjadi.

Tantangan 2: Bounded Concurrency Rate Limiter untuk Third-Party API (Semaphore)

Ketika backend aplikasi kita berintegrasi dengan pihak ketiga (seperti Payment Gateway atau API SMS), pihak ketiga sering membatasi jumlah koneksi paralel bersamaan (misal: maksimum hanya boleh ada 3 koneksi API paralel yang terhubung dalam satu waktu dari server kita). Membiarkan backend menembak ratusan API paralel akan membuat server kita diblokir.

- **Tugas:**

1. Gunakan Buffered Channel bertipe kosong `chan struct{}` bertindak sebagai **Semaphore** pembatas derajat paralelisme.
2. Buat channel semaphore dengan kapasitas buffer 3:

```
sem := make(chan struct{}, 3)
```

3. Sebelum memulai request HTTP ke API eksternal pihak ketiga di dalam kode Anda, lakukan "akuisisi token" dengan memasukkan struct kosong ke channel:

```
sem ← struct{}{} // Mengisi buffer. Jika terisi 3, pengirim berikutnya akan terblokir (mengantri)
```

4. Setelah request HTTP selesai (sukses maupun gagal), lepaskan token dari buffer agar slot dapat digunakan oleh antrian berikutnya:

```
←sem // Mengosongkan 1 slot buffer
```

5. Terapkan teknik *Bounded Concurrency* ini untuk membungkus pemanggilan API luar agar server Anda aman dari risiko pemblokiran akibat melebihi kuota paralel eksternal.